Demo-Modus

Lernunterlage: Fakes statt echter Clients, Dependency Injection je nach Einstellung, keine echten Daten auf dem Beamer

1 Worum es geht

Für die Präsentation gibt es eine **zweite Instanz** von Home Control: `./demo-starten.sh`, Adresse `http://localhost:5300`, Konto `demo@example.de`. Sie zeigt ein erfundenes Haus mit 5 Räumen und 12 Geräten, Ereignisse der letzten 7 Tage, schwankende Watt, eine Tür, die ab und zu aufgeht, ein Gerät offline, einen Akku bei 15 % und zwei Kameras mit selbst gezeichneten Bildern.

Warum keine echten Daten auf dem Beamer?

Grund	Erklärung
Datenschutz	Kamerabilder zeigen die eigene Wohnung, Gerätenamen und Zeiten verraten Gewohnheiten (wann jemand zu Hause ist).
Sicherheit	Keine echten Zugangsdaten im Spiel: Die Demo lädt keine User Secrets, ein Fehler kann nichts verraten.
Vorführeffekt	Schul-WLAN, Tuya-Störung oder Blink-Sperre lassen die echte App leer aussehen. Die Demo braucht kein Internet.
Akku und Kontingent	„Neues Bild“ kostet bei echten Kameras Akku, jeder Tuya-Abruf Kontingent. Im Demo kostet nichts etwas.
Wiederholbar	Jeder Start sieht gleich aus. Das Drehbuch der Vorführung passt immer.

2 Fakes statt echter Clients

Ein **Fake** ist eine einfache, funktionierende Ersatz-Implementierung, die sich nach außen genauso verhält wie das Original. Die Tests benutzen solche Fakes schon lange. Der Demo-Modus benutzt dieselbe Idee im laufenden Programm.

	Echter Betrieb	Demo-Modus	Ersetzt auf Ebene
Tuya	TuyaClient → Internet	TuyaClient → DemoTuyaHandler	HTTP (HttpMessageHandler)
Blink	BlinkHttpCameraClient	DemoBlinkCameraClient	Interface IBlinkCameraClient
Daten	homecontrol.db	homecontrol-demo.db	eigene Datei

- **Tuya auf der HTTP-Ebene:** Der echte `TuyaClient` signiert, holt ein Token und fragt den Status ab, nur antwortet statt der Cloud ein `HttpMessageHandler` mit erfundenen Werten im Tuya-Format. So laufen Abgleich, Messwerte, Skalierung (1234 → 123,4 W) und Online-LEDs als echter Code.
- **Blink auf der Interface-Ebene:** Alles hinter `IBlinkCameraClient` wird ersetzt. Endpunkte, Rate Limit und Oberfläche sehen keinen Unterschied.
- **Mock vs. Fake:** Ein Mock prüft im Test, ob er richtig aufgerufen wurde. Ein Fake *funktioniert* einfach, deshalb taugt er auch für eine Vorführung.

```
// Fake-Cloud: beantwortet die Anfragen des echten TuyaClient selbst
public class DemoTuyaHandler(TimeProvider time) : HttpMessageHandler
{
    protected override Task<HttpResponseMessage> SendAsync(HttpRequestMessage request, ...)
    {
        if (path == "/v1.0/token") return Ok(new { access_token = "demo-token", expire_time = 7200 });
        // ... Geräteliste mit online true/false, Status mit switch_1, cur_power, ...
    }
}
```

3 Dependency Injection je nach Einstellung

Program.cs entscheidet **einmal beim Start**, welche Klasse hinter einem Interface steckt. Wer den Dienst benutzt (Endpunkt, Abgleich), bekommt ihn per Konstruktor und fragt nie nach, ob es ein Fake ist.

```
var demoMode = DemoMode.IsEnabled(builder.Configuration);   // HomeControl:DemoMode
if (demoMode)
{
    tuyaHttpClient.ConfigurePrimaryHttpMessageHandler(sp => new DemoTuyaHandler(...));
    services.AddSingleton<IBlinkCameraClient, DemoBlinkCameraClient>();
}
else
{
    services.AddSingleton<IBlinkCameraClient, BlinkHttpCameraClient>();
}
```

- **Ein Schalter, eine Stelle:** Im restlichen Code gibt es kein `if (demo)`. Das hält ihn sauber und verhindert, dass die Demo etwas anderes vorführt als die echte App.
- **ConfigurePrimaryHttpMessageHandler:** Der „letzte“ Handler eines HttpClient ist normalerweise der, der ins Netzwerk geht. Hier ist es die Fake-Cloud.
- **Gleiche Lebensdauer:** Fake und echter Blink-Client sind beide Singletons. So bleibt z. B. das Bild nach „Neues Bild“ erhalten.

4 Sicherheitsnetz: nie versehentlich an

Die gefährlichste Panne wäre: Der Dienst läuft mit Fake-Daten, oder die Demo mit echten Zugangsdaten. `DemoMode.EnsureConsistent` prüft deshalb ganz am Anfang von Program.cs. Passt etwas nicht, startet die App gar nicht (**fail fast**):

1. DemoMode nur in der Umgebung `Demo`. Production oder Development mit DemoMode: Abbruch.
2. Umgebung Demo nur mit DemoMode.
3. Im Demo keine User Secrets (dort liegen Tuya-, Blink- und JWT-Schlüssel).
4. Im Demo muss die Datenbank „demo“ im Namen haben. Die echte homecontrol.db bleibt unberührt.
5. Demo-Passwort muss gesetzt sein.

- **Umgebung Demo:** ASP.NET Core lädt dann automatisch `appsettings.Demo.json`. Der Dienst (Production) lädt diese Datei nie.
- **.invalid:** Die Tuya-Adresse in der Demo endet auf `.invalid`, eine Endung, die es nie geben wird. Selbst ohne Fake ginge keine Anfrage an Tuya.
- **Wegwerf-Schlüssel:** Das Skript erzeugt bei jedem Start einen zufälligen JWT-Schlüssel.

> Gefährliche Einstellungen nicht nur dokumentieren, sondern beim Start prüfen: Lieber startet die App gar nicht als mit der falschen Konfiguration.

5 Werte, die schwanken und trotzdem testbar sind

Das Verhalten der Geräte steht in `DemoBehavior.StateAt(gerät, uhrzeit)`, einer **reinen Funktion**: gleiche Uhrzeit, gleiches Ergebnis. Die Schwankung kommt nicht aus `Random`, sondern aus einer Prüfsumme über Gerät und Minute.

Gerät	Regel
Haustür	alle 20 Minuten für 5 Minuten offen; der Abgleich fragt alle 3 Minuten, erwischt also jedes Öffnen
Kaffeemaschine	alle 30 Minuten 4 Minuten an, dann ca. 1,2 kW
Kühlschrank	immer an, Kompressor 8 von 20 Minuten (ca. 92 W, sonst 1,5 W)
Computer	tagsüber an, 75–145 W
Leselampe	abends an, tagsüber ab und zu, damit in der Vorführung etwas passiert
Drucker	offline (ausgesteckt): rote LED, keine alten Messwerte

Warum keine echte Zufallszahl? `string.GetHashCode()` und `Random` liefern bei jedem Start etwas anderes. Ein Test könnte dann nur „irgendwas“ prüfen. Mit der festen Prüfsumme (FNV-1a) prüft der Test genau: In einer Stunde ist die Tür 15 Minuten offen.

6 Beispielhaus und Verlauf

- `DemoHouse` beschreibt Räume und Geräte **einmal**. Seeder und Fake-Cloud lesen beide von dort, Namen und erfundene Tuya-Ids passen deshalb immer zusammen.
- `DemoDataSeeder` legt Ereignisse der letzten 7 Tage nach einem Tagesablauf an (Licht morgens und abends, Tür 5–9 Mal, Temperatur viermal am Tag). `new Random(2026)`: fester Startwert, also bei jedem Anlegen dieselben Ereignisse.
- Das Skript löscht die Demo-Datenbank bei jedem Start. So reichen die „letzten 7 Tage“ immer bis heute.
- Das Demo-Konto wird bei jedem Start auf das Passwort aus dem Skript gesetzt, eine Sperre wird aufgehoben.

Gefundener Fehler: Im ersten Versuch fehlte der Kühlschrank. EF Core speichert nur Objekte, die man hinzufügt *oder* die über eine Verknüpfung erreichbar sind. Die Geräte hingen nur an den Ereignissen, der Kühlschrank hat aber keine (er ist immer an). Lösung: Räume und Geräte ausdrücklich mit `AddRange` hinzufügen. Ein Test zählt jetzt 12 Geräte.

Zweiter Fehler: Die erfundenen Tuya-Ids hießen zuerst `demo-computer`. Echte Tuya-Ids bestehen aber nur aus Buchstaben und Ziffern, und das Speichern eines Geräts prüft das. Folge: In der Demo ließ sich kein Gerät bearbeiten. Lehre: Fake-Daten müssen dieselben Regeln erfüllen wie echte. Jetzt heißen sie `democomputer`, und ein Test speichert jedes Demo-Gerät einmal.

7 Läuft neben dem Dienst

	Dienst	Entwicklung	Demo
Start	dienst-einrichten.sh	app-starten.sh	demo-starten.sh
Port	5200	5111	5300
Umgebung	Production	Development	Demo
Datenbank	homecontrol.db	homecontrol.db	homecontrol-demo.db
Geheimnisse	User Secrets	User Secrets	keine

Eigener Ordner, eigene Datenbank, eigener Port: Die Regel „ein Prozess pro Datenbank“ bleibt gewahrt. Die Demo spricht nie mit Blink, das echte Blink-Token ist also nie in Gefahr.

8 Tests

Test	prüft
DemoMode_im_Dienst_oder_beim_Entwickeln_verhindert_den_Start	Production/Development mit DemoMode: kein Start
Demo_mit_echter_Datenbank_oder_User_Secrets_…	homecontrol.db oder User Secrets: kein Start
Dienst_und_Entwicklung_schalten_DemoMode_nirgends_ein	appsettings.json, …Development.json, dienst-einrichten.sh
Demo_startet_mit_5_Raeumen_12_Geraeten_und_7_Tagen_…	Beispielhaus und Verlauf
Abgleich_liest_die_Fake_Cloud_…	Watt, Akku 15 %, Drucker offline, Status-LEDs
Tuer_geht_ab_und_zu_auf_…	3-Minuten-Takt erwischt jedes Öffnen
Neues_Bild_im_Demo_…	anderes Bild, Aufnahmezeit jetzt

9 Kurzantworten für Rückfragen

Frage	Antwort
Ist das echt?	Nein, Beispieldaten. Das Banner sagt es. Der Code ist derselbe wie im echten Betrieb.
Was ist ein Fake?	Eine einfache, funktionierende Ersatz-Implementierung mit derselben Schnittstelle.
Warum nicht einfach die echte App?	Datenschutz (Wohnung, Gewohnheiten), kein Internet nötig, kein Akku, wiederholbar.
Wie wird umgeschaltet?	Dependency Injection: Program.cs registriert je nach HomeControl:DemoMode echt oder Fake.
Kann der Dienst aus Versehen Demo sein?	Nein: DemoMode nur in der Umgebung Demo, sonst startet die App nicht. Getestet.
Warum Tuya auf HTTP-Ebene?	Damit Abgleich, Messwerte und LEDs echter Code bleiben, nur die Cloud ist erfunden.
Woher kommen die Bilder?	Selbst mit Pillow gezeichnet, Aufschrift DEMO-BILD, keine Fotos aus dem Netz.

Frage	Antwort
Warum schwanken die Werte nicht zufällig?	Feste Prüfsumme statt Random: sieht lebendig aus und bleibt testbar.

10 Merksätze

- Auf den Beamer gehören Beispieldaten, nie echte Kamerabilder, Namen oder Zugangsdaten.
- Fake = gleiche Schnittstelle, einfache Implementierung. Umschalten nur an EINER Stelle (DI).
- Gefährliche Schalter beim Start prüfen (fail fast), nicht nur dokumentieren.
- Eigene Datenbank, eigener Port, keine Secrets: Demo und Dienst stören sich nicht.
- Reine Funktionen statt Zufall: lebendige Werte, die sich trotzdem testen lassen.